

Reinforcement Learning–Based Adaptive Traffic Signal Control in a Simulated Intersection

Bassam Walid
55-0499

Youssef Mohamed
55-1031

Styven Hany
55-10067

Ali Khaled
55-6888

Somaya Magdy
55-5027

I. ABSTRACT

This project investigates reinforcement-learning-based adaptive traffic signal control for a simulated single four-way intersection. The intersection is modeled as a Markov Decision Process where the controller observes traffic queues, signal phase information, and timing features, then selects which direction receives the green signal. A Python queue-based simulator was developed with stochastic Poisson vehicle arrivals, yellow-light transitions, finite queue capacity, and multiple traffic scenarios including low uniform, medium uniform, high uniform, uneven, and rush-hour demand. A fixed-time controller was compared with tabular Q-learning, tabular SARSA, and a Deep Q-Network using a continuous-observation representation. A reward-selection experiment was conducted, and the final reward was chosen as a normalized composite reward including queue change, waiting time, throughput, and a switching penalty. Final results showed that all reinforcement learning controllers outperformed fixed-time control, with DQN achieving the highest overall score and lowest queue/waiting time, while Q-learning was the best tabular method. The project demonstrates that reinforcement learning can improve traffic signal control, while also highlighting the importance of reward design, phase-switching behavior, and testing under different traffic conditions.

II. AIM OF THE PROJECT

The aim of this project is to design, implement, and evaluate an adaptive traffic signal control system for a simulated single four-way intersection using reinforcement learning. Instead of relying on a fixed-time signal cycle, the controller learns to make decisions based on the observed traffic conditions. The main objective is to reduce congestion and waiting time, maintain high vehicle throughput, and avoid unrealistic excessive switching between traffic phases. The project focuses on studying how different reinforcement learning methods behave when applied to the same traffic-control problem. For this reason, Q-learning and SARSA were implemented as tabular reinforcement learning methods using a discrete state representation. In addition, a Deep Q-Network was implemented as an advanced extension using a continuous-observation state representation. A fixed-time controller was also included as a baseline so that the improvement achieved by the learning-based controllers could be measured directly.

Another important aim of the project is to investigate how reward-function design affects the behavior of the learned controller. Several reward formulations were tested and

compared before selecting the final reward function. The selected reward balances queue reduction, waiting-time reduction, throughput improvement, and phase-switching control. This allows the controller to improve traffic performance while avoiding unrealistic behavior such as switching phases too frequently.

The final goal is not only to obtain a controller with good numerical performance, but also to understand the strengths and limitations of each method. The project compares the controllers under multiple traffic scenarios, including uniform traffic, uneven traffic, and rush-hour demand. This makes it possible to evaluate whether the learned policies are robust under different traffic conditions and to analyze the tradeoff between performance, stability, and interpretability.

III. ENVIRONMENT FORMULATION

The simulation environment represents a single isolated four-way intersection with traffic arriving from the North, South, East, and West directions. At each timestep, vehicles arrive according to stochastic Poisson arrival rates, and the controller selects which direction should receive the green signal. The action space contains four possible actions, where each action corresponds to activating one green phase: North, South, East, or West. Only one direction can be served at a time, while all other directions remain red. If the controller selects a different phase from the current one, a yellow-light transition of fixed duration is applied before the new green phase becomes active. This models the delay caused by phase switching and discourages unrealistic rapid switching behavior.

The environment is queue-based. Each direction has a queue that stores the number of vehicles waiting. During every timestep, new vehicles are added to each queue based on the selected traffic scenario, and vehicles are removed from the queue of the active green direction according to the service rate. In the final implementation, the service rate is 2 vehicles per timestep, the yellow duration is 2 timesteps, the maximum queue length per direction is 40 vehicles, and each episode lasts up to 500 timesteps unless an overflow condition occurs first. An episode terminates when the maximum timestep limit is reached or when any queue reaches the maximum allowed queue length. This termination condition makes the environment suitable for evaluating both normal traffic performance and failure cases under heavy congestion.

The traffic-control problem is formulated as a Markov Decision Process. The state describes the current traffic condition, the action represents the selected green phase, the transition is produced by vehicle arrivals, vehicle departures, yellow-light timing, and queue updates, and the reward

evaluates how good the controller's action was. The final reward is a normalized composite reward that considers queue change, total waiting time, throughput, and a switching penalty. The objective of the agent is to learn a policy that selects traffic-light phases in a way that reduces congestion and waiting time, increases throughput, and avoids unnecessary phase changes.

A. Discrete Formulation

The discrete formulation is used for the tabular Q-learning and SARSA agents. In this formulation, the exact queue lengths are converted into discrete bins to keep the state space compact and suitable for a Q-table. Each of the four queue lengths is classified into one of three levels: low, medium, or high. A queue length from 0 to 3 vehicles is considered low, a queue length from 4 to 7 vehicles is considered medium, and a queue length of 8 or more vehicles is considered high. This discretization reduces the number of possible states while still preserving enough information about the traffic conditions.

The discrete state is represented by the queue bin of each direction, the current active phase, and the phase timer bin. Therefore, the tabular state has the form: north queue bin, south queue bin, east queue bin, west queue bin, current phase, and phase timer bin. The phase timer is also discretized to describe how long the current phase has been active. A short phase duration corresponds to 0 to 5 timesteps, a medium duration corresponds to 6 to 15 timesteps, and a long duration corresponds to 16 or more timesteps. Including the phase timer helps the agent learn whether it should keep serving the current direction or switch to another one.

The action space in the discrete formulation contains four actions. Action 0 gives the green signal to the North direction, action 1 gives the green signal to the South direction, action 2 gives the green signal to the East direction, and action 3 gives the green signal to the West direction. Q-learning and SARSA both use this same state and action representation, which makes the comparison between the two algorithms fair. The main difference between them is in the learning update. Q-learning is off-policy because it updates using the best estimated next action, while SARSA is on-policy because it updates using the action that is actually selected next under the current exploration policy.

This discrete formulation is simple, interpretable, and well suited for tabular reinforcement learning. Its main advantage is that the learned Q-table can be understood more easily because each state corresponds to meaningful traffic conditions such as low, medium, or high queues. However, the disadvantage is that discretization loses some information. For example, queues of 8 and 30 vehicles are both treated as "high," even though they represent very different congestion levels. This limitation motivated the use of a continuous-observation formulation for the DQN extension.

B. Continuous Formulation

The continuous formulation is used for the Deep Q-Network agent. In this formulation, the same traffic environment dynamics are used, but the state representation is changed from discrete bins to a normalized continuous observation vector. This means that DQN does not use a different simulator; it receives a richer version of the same traffic state.

The actions, reward function, vehicle arrivals, yellow-light transitions, service rate, and termination conditions remain the same as in the discrete environment.

The DQN observation vector contains 19 values. These include the four queue lengths normalized by the maximum queue size, a one-hot representation of the current active phase, a one-hot representation of the pending phase during yellow transitions, the normalized phase timer, the normalized yellow timer, the normalized episode progress, and the four normalized arrival rates. This gives the neural network more detailed information than the tabular state because it can observe the actual relative queue sizes instead of only low, medium, and high bins.

The continuous-observation formulation is better suited for neural networks because DQN can approximate the action-value function without needing to store every state-action pair in a table. The neural network takes the 19-dimensional observation as input and outputs four Q-values, one for each possible traffic-light action. The action with the highest Q-value is selected during greedy evaluation. During training, DQN uses experience replay, a target network, gradient-based optimization, and epsilon-greedy exploration to learn stable action-value estimates.

It is important to clarify that this formulation is continuous in terms of state observation, not in terms of the action. The action space is still discrete because the controller still chooses one of four possible green phases. Therefore, this extension is best described as a continuous-observation DQN formulation rather than full continuous control. A true continuous-control version would require the agent to output continuous values such as green-light duration or phase extension time. In this project, the continuous formulation was used to test whether a richer state representation and neural function approximation could improve traffic signal control compared to the compact tabular representation.

IV. SCORE CALCULATION

To compare the controllers, a combined score was calculated relative to the fixed-time baseline. The score measures the percentage improvement in queue length, waiting time, and throughput, while penalizing excessive phase switching. It was calculated as:

$$queue_{improvement} = \frac{(fixed_{avgqueue} - controller_{avgqueue})}{fixed_{avgqueue}}$$

$$wait_{improvement} = \frac{(fixed_{avgwait} - controller_{avgwait})}{fixed_{avgwait}}$$

$$throughput_{improvement} = \frac{(controller_{Tp} - fixed_{Tp})}{fixed_{Tp}}$$

$$switch_{penalty_{score}} = \frac{(controller_{switches} - fixed_{switches})}{fixed_{switches}}$$

$$score_{percent} = 100 * [\\ 0.40 * queue_{improvement} \\ + 0.40 * wait_{improvement} \\ + 0.20 * throughput_{improvement} \\ - 0.10 * switch_{penalty_{score}}]$$

The weights were selected to reflect the main priorities of the traffic-control task. Queue length and waiting time were each given the highest weight of 0.40 because reducing congestion and reducing how long vehicles wait are the main objectives of the controller. Throughput was given a weight of 0.20 because clearing more vehicles is important, but it should not come at the cost of very long queues or waiting times. The switching term was given a penalty weight of 0.10 to discourage unrealistic rapid phase changes while still allowing the controller to switch when needed. Negative scores were allowed, since a negative value indicates that the controller performed worse than the fixed-time baseline according to the combined metric.

V. REWARD EXPERIMENT

Before final training, a reward-function experiment was performed to select a suitable reward for the traffic signal controller. The purpose of this experiment was to compare different reward formulations and determine which one produced the best balance between queue reduction, waiting-time reduction, throughput improvement, and realistic phase-switching behavior. The rewards were evaluated using the score percentage defined relative to the fixed-time baseline. Negative scores were allowed, since a negative score means that the controller performed worse than fixed-time control according to the combined metric.

The first stage of the experiment compared the main reward formulations R1 to R8. The results are shown in Fig.1. The single-objective waiting-time and throughput rewards, R2 and R3, performed worse than the fixed-time baseline, which shows that optimizing only one objective can lead to poor overall behavior. The un-normalized composite reward R6 also performed poorly, which indicates that reward terms with different numerical scales should not be directly combined without normalization. The best Stage A reward was R7, the normalized composite reward, which achieved the highest score from R1 to R8. This shows that combining queue change, waiting time, and throughput using normalized weights produced a more balanced learning signal than using any single term alone.

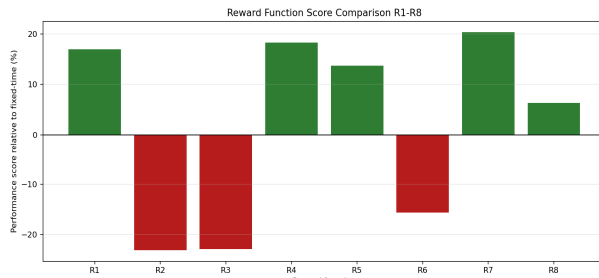


Figure 1

Reward	Formula / Meaning	Score %
R1	-queue_delta	16.97
R2	-total_waiting_time	-23.15
R3	throughput	-22.93
R4	-queue_delta - w2 * total_waiting_time	18.34

R5	-queue_delta + w3 * throughput	13.77
R6	-queue_delta - total_waiting_time + throughput	-15.65
R7	-w1 * queue_delta - w2 * total_waiting_time + w3 * throughput	20.38
R8	-0.5 * total queue + 2.0 * throughput - max_red_queue - switched	6.34

The normalized composite reward R7 was calculated using queue change, total waiting time, and throughput. Since these components have different numerical scales, inverse-scale normalization was used so that no term dominates only because its values are numerically larger. The component scales were first calculated from the experiment data, then converted into weights as follows:

$$\begin{aligned} inv_q &= 1 / (\text{mean}(\text{abs}(\text{queue_delta})) + \text{eps}) \\ inv_w &= 1 / (\text{mean}(\text{abs}(\text{total_waiting_time})) + \text{eps}) \\ inv_t &= 1 / (\text{mean}(\text{abs}(\text{throughput})) + \text{eps}) \end{aligned}$$

$$\begin{aligned} w1 &= inv_q / (inv_q + inv_w + inv_t) \\ w2 &= inv_w / (inv_q + inv_w + inv_t) \\ w3 &= inv_t / (inv_q + inv_w + inv_t) \end{aligned}$$

$$\begin{aligned} w1 &= 0.4843 \\ w2 &= 0.0169 \\ w3 &= 0.4989 \end{aligned}$$

The second stage of the reward experiment added a switch penalty to the Stage A winner, R7. This was done because frequent phase switching can reduce realism and cause unnecessary yellow-light delays. The switch-penalty sweep is shown in Fig.2. Adding a moderate switch penalty improved the score compared to using no switch penalty. Very large switching penalties reduced performance, because the controller became less flexible and could not react as effectively to changing traffic conditions.

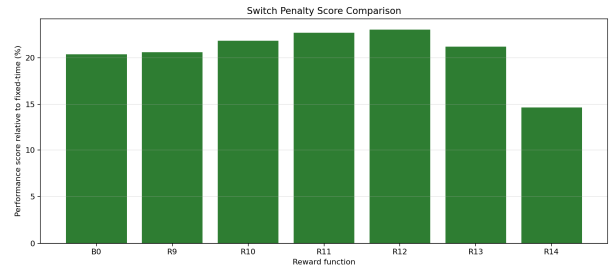


Figure 2

Reward	Formula	Score (%)
B0	R7 - 0.00 * switched	20.38
R9	R7 - 0.10 * switched	20.62
R10	R7 - 0.25 * switched	21.85
R11	R7 - 0.50 * switched	22.70
R12	R7 - 1.00 * switched	23.06
R13	R7 - 1.25 * switched	21.20
R14	R7 - 2.00 * switched	14.65

Although the highest score in the switch-penalty sweep was obtained at a penalty of 1.00, the penalty of 0.50 produced a very close result. Since the final controller would later be trained on mixed scenarios the final switching penalty was selected as the average between 0.50 and 1.00. Therefore, the final reward used for the rest of the project was:

$$\begin{aligned}
 R_{final} = & \\
 & -0.4843 * queue_delta \\
 & -0.0169 * total_waiting_time \\
 & +0.4989 * throughput \\
 & -0.75 * switched
 \end{aligned}$$

VI. APPLIED RL METHODS

Three reinforcement learning methods were applied in this project: Q-learning, SARSA, and DQN. Q-learning and SARSA were used as tabular methods with the discrete state formulation, where the learned values are stored in a Q-table for each state-action pair. Q-learning is an off-policy method, meaning it updates its Q-values using the best estimated next action, while SARSA is an on-policy method, meaning it updates using the action actually selected by the current exploration policy. This makes Q-learning more aggressive in learning the optimal greedy policy, while SARSA usually reflects a more cautious learning behavior during exploration. DQN was implemented as an advanced extension using the continuous-observation formulation. Instead of storing values in a Q-table, DQN uses a neural network to approximate the Q-values for the four possible traffic-light actions. It takes the 19-dimensional observation vector as input and outputs one Q-value for each action. Experience replay and a target network were used to improve training stability. All learned agents used epsilon-greedy exploration during training and greedy action selection during final evaluation.

VII. MODELS TRAINING

After selecting the final reward function, the reward was fixed and used for training all learning-based controllers. The final training stage was performed using mixed traffic scenarios, meaning that each training episode was sampled from one of the predefined traffic conditions: low uniform, uniform medium, high uniform, uneven, or rush-hour traffic. This was done to expose the agents to different traffic demand patterns instead of training them only on one fixed scenario. The training process used a fixed base seed of 42, while the episode seed was increased with the episode index. This made the training process reproducible while still allowing each episode to contain a different stochastic traffic pattern. Q-learning and SARSA were trained using the same discrete environment, the same final reward function, and the same hyperparameters. Both methods used epsilon-greedy exploration during training, where the agent initially explores the action space and gradually shifts toward exploiting the learned Q-values. The exploration rate started at 1.0 and decayed until it reached a minimum value of 0.01. Both tabular agents were trained for 12000 episodes with a learning rate of 0.1 and a discount factor of 0.95. Using the same training setup for Q-learning and SARSA made the comparison between the two tabular methods fair, since the main difference between them was the update rule rather than the training conditions.

The Deep Q-Network was trained using the continuous-observation environment. Unlike the tabular agents, DQN does not store a Q-table. Instead, it uses a neural network to approximate the action-value function. The DQN model receives a 19-dimensional normalized observation vector as input and outputs four Q-values, one for each possible traffic-light phase. The DQN was trained for 8000 episodes using experience replay, a target network, batch updates, and gradient clipping to improve stability. CUDA GPU acceleration was used for the neural network updates, while the traffic simulation itself still ran on the CPU.

The fixed-time controller was not trained because it is a rule-based baseline. It cycles through the four directions with a fixed green duration of 20 timesteps and does not observe the traffic state. Its purpose is to provide a reference point for measuring the improvement achieved by the learning-based controllers.

The training curves were plotted using rolling averages because the raw training data is naturally noisy. This noise is caused by stochastic vehicle arrivals, epsilon-greedy exploration, and mixed-scenario training, where different episodes may have different traffic difficulty levels. Rolling averages make the learning trend easier to interpret without changing the raw training data saved. A rolling window of 500 episodes was used for Q-learning and SARSA, while a rolling window of 200 episodes was used for DQN.

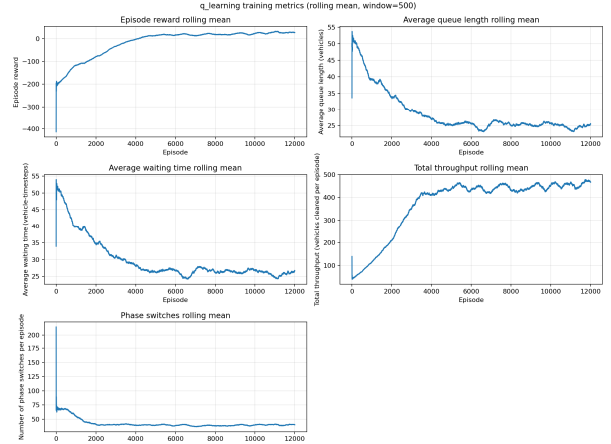


Figure 3

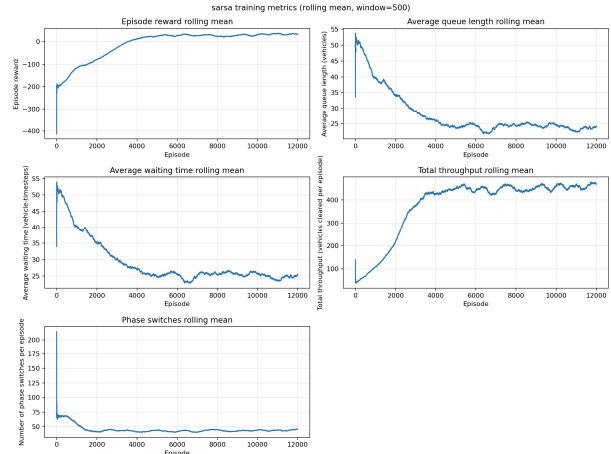


Figure 4

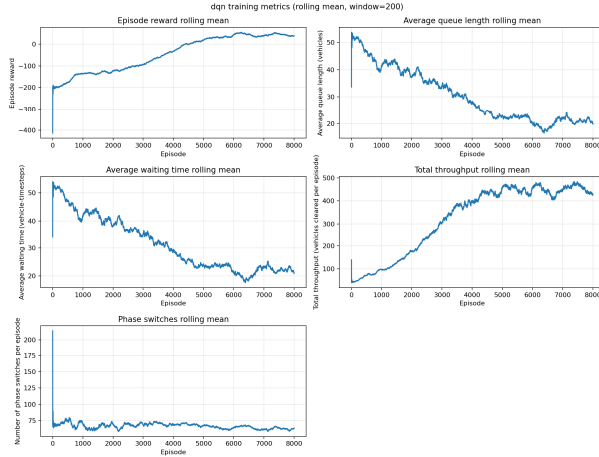


Figure 5

The training curves are used only to show whether the agents learned more stable behavior over time. The final judgment of controller performance is not based on training reward alone, because training still includes exploration and mixed episode difficulty. Therefore, the final comparison between controllers was performed separately using greedy evaluation on fixed scenario test sets.

VIII. RESULTS

The final controllers were evaluated across five traffic scenarios: low uniform, uniform medium, high uniform, uneven, and rush-hour traffic. Each controller was evaluated using three independent evaluation seed sets, with 100 episodes per seed and scenario. This gives a total of 1500 evaluation episodes per controller. The evaluation was performed greedily for the trained reinforcement learning agents, meaning that exploration was disabled and each agent selected the action with the highest learned value.

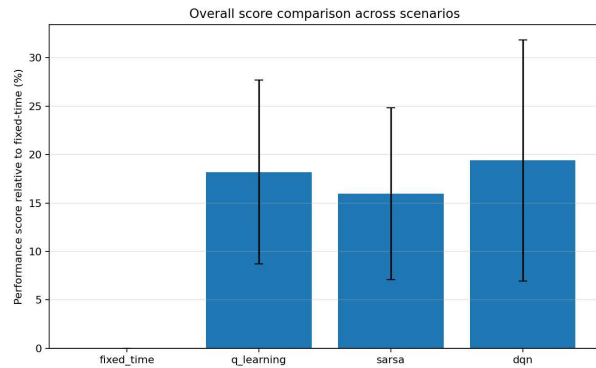


Figure 6

Fig.6 shows the overall performance score of each controller relative to the fixed-time baseline. The fixed-time controller has a score of 0% because it is used as the reference baseline. All learning-based controllers achieved a positive score, showing that reinforcement learning improved the traffic signal behavior compared to fixed-time control. DQN achieved the highest overall score, followed by Q-learning and then SARSA. The vertical black lines represent the

standard deviation of the controller's scores across the different scenarios. A larger vertical line means that the controller's performance varied more between scenarios. DQN has the largest variation, which shows that although it performed best overall, it was less consistent across the different traffic conditions.

Metric	Fixed-time	Q-learning	SARSA	DQN
Overall Score (%)	0.00	18.19	15.94	19.39
Score Std	0.00	9.49	8.87	12.45
Avg Queue	35.07	24.53	23.27	17.91
Avg Wait	36.01	25.67	24.42	19.08
Throughput	246.96	458.76	459.63	465.01
Switches	12.47	38.24	44.10	59.59
Overflow Episodes	893	307	303	300

The overall results show that DQN achieved the best average performance, with the highest score, the lowest average queue length, the lowest average waiting time, and the highest throughput. However, DQN also produced the highest number of phase switches. This means that DQN was more aggressive in changing traffic phases. The switch penalty in the score reduces the benefit of this behavior, so even when DQN improves queue length and waiting time, its score does not increase as much as it would if switching were ignored. Therefore, DQN provides the best traffic performance overall, but it is less smooth and less consistent than the tabular methods.

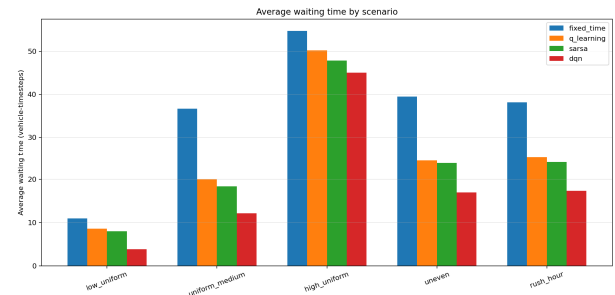


Figure 7

Fig.7 compares the average waiting time for each controller in each scenario. The learning-based controllers reduced waiting time compared to fixed-time control in all scenarios. The improvement is especially clear in the uniform medium, uneven, and rush-hour scenarios, where fixed-time control cannot adapt to the changing traffic demand. DQN achieved the lowest waiting time in most scenarios, especially in low uniform, uniform medium, uneven, and rush-hour traffic. In the high uniform scenario, all controllers had high waiting time because the traffic demand was heavy from all directions, making the intersection close to saturation.

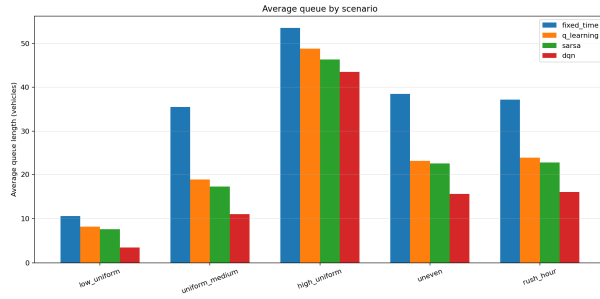


Figure 8

Fig.8 shows the average queue length for each controller. The trend is similar to the waiting-time results because larger queues usually lead to longer waiting times. All reinforcement learning controllers reduced average queue length compared to the fixed-time baseline. DQN achieved the lowest queue length in most scenarios, showing that the continuous-observation representation helped it react strongly to queue buildup. Q-learning and SARSA also improved the queue length significantly compared to fixed-time control, but their reductions were generally smaller than DQN.

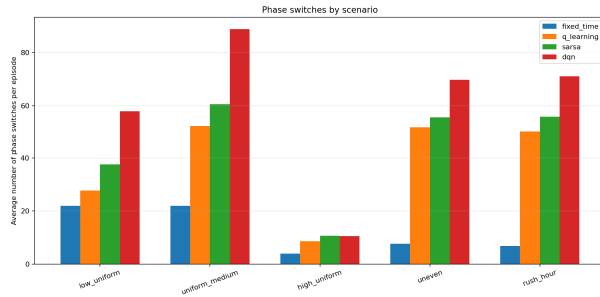


Figure 9

Fig.9 shows the average number of phase switches per episode. Fixed-time control has the lowest number of switches because it follows a fixed cycle and does not react to traffic conditions. The reinforcement learning controllers switch more often because they adapt their decisions based on the observed traffic state. DQN switches the most in almost all scenarios, which explains why its score is penalized despite its strong queue and waiting-time performance. Q-learning switches less than SARSA overall, which helps it achieve a higher final score even though SARSA has slightly lower average queue and waiting time.

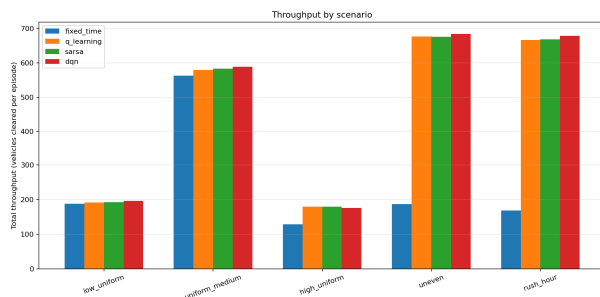


Figure 10

Fig.10 compares the total throughput, which represents the number of vehicles cleared per episode. The reinforcement learning controllers generally achieved higher throughput than fixed-time control, especially in uneven and rush-hour scenarios. This shows that the learned policies were better at serving vehicles when demand was not evenly distributed. DQN achieved the highest overall throughput, but the difference between Q-learning, SARSA, and DQN was smaller than the difference between the learning-based controllers and fixed-time control.

Scenario	Fixed-time	Q-learning	SARSA	DQN
Low Uniform	0.00	15.47	15.08	37.26
Uniform Medium	0.00	23.68	23.67	24.81
High Uniform	0.00	2.71	1.26	5.23
Uneven	0.00	25.67	21.89	18.61
Rush Hour	0.00	23.41	17.79	11.04
Overall	0.00	18.19	15.94	19.39

The scenario score table shows that the best controller depends on the traffic condition. DQN performed very strongly in low uniform and uniform medium traffic, and it was also the best in high uniform traffic, although all controllers struggled in that scenario. Q-learning performed better than DQN in the uneven and rush-hour scenarios because it achieved a better balance between traffic improvement and switching behavior. SARSA was competitive, but its overall score was reduced because it switched more than Q-learning and had lower scenario scores in uneven and rush-hour traffic.

For the tabular methods, Q-learning was the final winner based on the overall score. SARSA achieved slightly lower average queue and waiting time, but Q-learning had fewer phase switches and a higher combined score. This shows that the best controller is not determined only by queue length or waiting time; switching behavior also affects the final evaluation because excessive switching can be unrealistic and causes additional yellow-light delays.

Overall, the results show that reinforcement learning clearly improved traffic signal control compared to fixed-time operation. DQN achieved the best overall traffic performance, but with higher switching and higher variability. Q-learning provided the best balance among the tabular methods, while SARSA gave smoother congestion metrics but a lower final score. The high uniform scenario remained the most difficult for all controllers because all directions had heavy demand, leaving limited capacity for the signal controller to prevent queue buildup.

IX. VISUALIZATION

To make the learned traffic-light behavior easier to understand, an improved visualization tool was added to the project. The visualization does not change the environment, training process, reward function, or learned policies. Its purpose is only to display the controller behavior in a clearer way for analysis and presentation. The visualizer shows the four-way intersection layout, the currently active traffic phase, vehicle queues in each direction, traffic-light colors, queue bars, and a metrics panel containing information such as the controller name, scenario, timestep, active phase, total

queue, waiting time, reward, throughput, and number of switches.

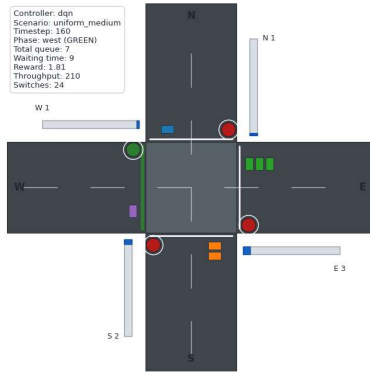


Figure 11

The included snapshot, fig.11, shows one example rollout for the DQN controller under the uniform_medium scenario. The green traffic light indicates the currently active served direction, while red lights indicate stopped directions. The small colored vehicle blocks represent queued vehicles waiting at each approach, and the side bars show the queue size for each direction. The metrics box in the upper-left corner provides a compact summary of the current simulation state, including the total queue, waiting time, reward, throughput, and number of phase switches at that timestep. This makes it easier to visually compare how each controller manages queues and switches phases during an episode.

In addition to static snapshots, GIF animations were generated for the different controllers. These animations show the evolution of the traffic queues and signal phases over time, making the controller behavior easier to interpret than numerical metrics alone. The GIF files are included in the submitted code/results folder and can be viewed separately for the fixed-time, Q-learning, SARSA, and DQN controllers. They are useful for demonstrating how the reinforcement learning controllers respond to changing queue conditions compared to the fixed-time baseline.

Only one representative snapshot is included in the main report to avoid making the report too long. The full animations are provided in the project folder as supplementary visual results. This keeps the report focused on the main quantitative results while still showing that the project includes a visual method for inspecting policy behavior.

X. LIMITATIONS AND KEY INSIGHTS

The final results show that reinforcement learning improved traffic signal control compared to the fixed-time baseline, but the project still has some limitations. The simulation environment is queue-based and simplified, meaning it does not model detailed vehicle motion such as acceleration, braking, lane changing, turning movements, pedestrian crossings, or realistic road geometry. Each direction is mainly represented by a queue length, and the controller chooses

which direction receives the green signal. This makes the environment suitable for studying reinforcement learning methods, but it is less realistic than a microscopic traffic simulator such as SUMO.

Another limitation is the action space. The controller selects only one of four green phases: North, South, East, or West. In real intersections, traffic signals usually include more complex phase structures such as combined movements, protected turns, pedestrian timing, minimum green times, and safety constraints. Therefore, the learned policies are useful for comparing reinforcement learning methods, but more realistic signal rules would be needed before applying the approach to a real intersection.

One of the main challenges was reward design. Rewards based on a single objective, such as waiting time or throughput only, did not always produce good overall behavior. The reward experiment showed that a normalized composite reward was more effective because it balanced queue change, waiting time, and throughput without allowing one term to dominate due to its larger scale. Adding a switching penalty was also important because excessive phase switching can reduce realism and create extra yellow-light delays. This showed that the best reward should balance traffic performance with smooth signal behavior.

Training was also challenging because the agents learned under stochastic vehicle arrivals and mixed traffic scenarios. This made the raw training curves noisy, since some episodes were easier while others, such as high-uniform and rush-hour traffic, were more difficult. Rolling-average curves were therefore used to show the learning trend, while final performance was judged using greedy evaluation across fixed scenarios and evaluation seeds.

The high-uniform scenario remained difficult for all controllers because all directions had heavy demand and the intersection was close to saturation. This shows that reinforcement learning can improve decision-making, but it cannot overcome the physical capacity limit of the intersection. The comparison between algorithms also showed useful tradeoffs. DQN achieved the best overall score and the lowest average queue and waiting time, but it switched phases the most and had the largest variation across scenarios. Q-learning achieved the best score among the tabular methods, while SARSA gave slightly lower queue and waiting time but was penalized by more switching.

Overall, the main insight gained is that reinforcement learning can clearly outperform fixed-time control, but good performance depends strongly on the environment formulation, reward design, and evaluation method. A useful traffic controller should not only reduce queue length and waiting time, but also maintain high throughput, avoid excessive switching, and remain stable under different traffic conditions.

XI. CONCLUSION

This project successfully implemented and evaluated reinforcement-learning-based adaptive traffic signal control for a simulated four-way intersection. The traffic-control problem was formulated as a Markov Decision Process, where the controller observes the current traffic state and selects the green-light phase. A Python simulation environment was developed with stochastic vehicle arrivals,

yellow-light transitions, queue overflow conditions, and multiple traffic scenarios. The project compared a fixed-time baseline with Q-learning, SARSA, and DQN controllers. The results showed that all reinforcement learning controllers improved performance compared to the fixed-time controller. The learned agents reduced average queue length and waiting time while increasing throughput across most scenarios. The reward experiment also showed that using a normalized composite reward was more effective than relying on a single objective such as waiting time or throughput alone. Adding a moderate switching penalty further improved the behavior by discouraging unrealistic frequent phase changes. Among all controllers, DQN achieved the highest overall score and the lowest average queue and waiting time. This shows the advantage of using a richer continuous-observation representation with a neural network. However, DQN also produced the highest number of phase switches and had the largest variation across scenarios, meaning that it was the strongest overall but less stable. Among the tabular methods, Q-learning achieved the higher final score, while SARSA produced slightly lower queue and waiting-time values but was penalized by more switching. Overall, the project demonstrates that reinforcement learning can be an effective approach for adaptive traffic signal control. It also shows that controller performance depends not only on the learning algorithm, but also on the reward function, state representation, evaluation method, and traffic scenario. The final results confirm that adaptive learning-based control can outperform fixed-time control, while also highlighting the need to balance traffic efficiency with realistic and stable signal behavior.

XII. SUPPLEMENTARY FILES

The submitted project folder contains the complete source code, configuration files, saved models, training logs, evaluation logs, generated figures, and GIF animations. The main final outputs are stored in the `results/final_mixed_r11_gpu/` folder, including training curves, final comparison plots, scenario evaluation results, metric CSV files, model outputs, and improved traffic animations for the fixed-time, Q-learning, SARSA, and DQN controllers. These files are provided as supplementary material to support reproducibility and to allow the full experiments and visualizations to be inspected outside the written report.

XIII. REFERENCES

- [1] M. Guo, P. Wang, C. Y. Chan, and S. Askary, "A reinforcement learning approach for intelligent traffic signal control at urban intersections," 2019 IEEE Intelligent Transportation Systems Conference (ITSC), 2019, pp. 4242–4247.
- [2] S. M. M. R. Swapno et al., "Traffic light control using reinforcement learning," Bangladesh University of Business and Technology, 2023.
- [3] S. İlgen and A. Durdu, "Reinforcement learning based traffic signal control: A performance comparison under different traffic scenarios," The 14th International Symposium on Advanced Topics in Electrical Engineering (ATEE), 2025.
- [4] P. Michailidis, I. Michailidis, C. R. Lazaridis, and E. Kosmatopoulos, "Traffic signal control via reinforcement learning: A review on applications and innovations," *Infrastructures*, vol. 10, no. 114, 2025.